

what
is
slop-
ware

,
how
do
you
spot
it,
what
do
you
do?

a cur-
riculum
that
teaches
you to
measure
the dis-
tance
between
some-
thing
that
looks
like it
works
and
some-
thing
that ac-

tually
holds
up.
every
chapter
ends
with a
check
you run
on your
own
project.

6 / 55
chapters
· pdf ·
every
published
chapter
on one
page. for
printing
or the
pdf.

00 why
does
this
book
exist?
01 what
is the
thing
on your
screen?
02 why
can
your
deploy
not

find
your
files?

03 why
does
your
link
only
open
for
you?

04
what
is the
dif-
fer-
ence
be-
tween
npm
run
dev
and
npm
run
build?

05 why
does
the
till
stand
in a
dif-
ferent
room?

00

why does this book ex- ist?

Why this
book ex-
ists,
who is
writing
it, and
how to
read it.

I have
been using
forums
such as
Reddit,
Quora and
Ekşi
Sözlük for
years, and
once I was
one of the
people
posting
there.
People
make fun
of local-
host:3000
users, and
they be-
lieve CS is
dead be-
cause

somebody
developed
a local
website.

Actually, I
hate need-
less jokes
made just
to fit in.
But also,
lack of
technical
depth gave
those lo-
calhost:30
00 owners
the
courage of
a super-
hero. So
instead of
joining in,
I decided
to create a
blog about
how to
avoid slop-
ware,
based on
my own
experi-
ences.

Learnin
g it cost
me 53 re-
pos, 8 pull
requests
and 6
Claude
Max x20
accounts.
It also
cost me
exploding
token bills,
sleepless

nights,
and the 5
kilos I lost
to hunger
and lack
of sleep. I
do not
want that
to happen
to any-
body else,
and I do
not want a
power this
large to
stay hid-
den inside
one class
of people.
I am using
Claude
while I
write this,
and I am
sure Dario
Amodei
would
want the
same. So I
am writing
it with my
own
hands, out
of every-
thing I
pulled
from my
own pain.

I do
not believe
that CS
has been
replaced
by AI.*
Over time
there was

COBOL
and C++
did not re-
place it,
Java did
not re-
place
C++, and
Python
did not re-
place
Java.
Given the
nature of
high level
languages,
I treat an
LLM as
one more
high level
language.
(And
where has
engineer-
ing gone?
Engineer-
ing is the
difference
in the
thinking
itself, in
knowing
what to
make and
in con-
necting
technolo-
gies and
developing
them.)

I love
vibecod-

ing

Tony
Stark is a
vibecoder
with un-
limited to-
kens. As
far as I
see, in-
ternships
and
projects
are pre-
sented as
though
they be-
long to
certain
people.
What gets
inherited
is not only
money.
You can
call it in-
herited vi-
sion, you
can call it
social
climbing,
and there
is even a
niche for it
now,
something
like a class
based net-
working
hub
founder.
What will
happen
soon is
that some-
body

starts a
business
with a
Claude
Pro ac-
count and
has gradu-
ates of
good
schools
working
under-
neath
them,
when
those same
students
could have
started
that com-
pany with
their
pocket
money.
Neverthele
ss, if it is
not rocket
science
and it is
not tank
design, I
do not
think any
work that
needs no
large capi-
tal is as
big and as
irreplace-
able as it
is claimed
to be.
Your ap-
plication
was reject-
ed, so say

that you
could do
this too
and carry
on your
way.

Everyo
ne should
benefit
from tech-
nology,
and not
only for
business
and mon-
ey, but for
making
products
for your
own needs.
As I be-
lieve from
Plato, no
one does
wrong
willingly,
and no-
body
writes bad
software
on pur-
pose ei-
ther.

Outside
academic
purposes,
CS is no
different
from being
able to
sew a but-
ton on.
Everyone
should
learn it in
order to

make the
ideas they
dream of
real, and
most im-
portantly,
everyone
can, be-
cause I be-
lieve that
for once in
history
anyone
can build
anything.

What
matters is
whether
anything
stands be-
hind the
code.

Think
about the
last time
your ter-
minal told
you every
test
passed. It
told you
that some
tests ran
and none
of them
com-
plained,
and noth-
ing about
whether
they
would
have no-
ticed a
broken
app.

Who am
I?

I am
Damla, a
CS stu-
dent at
Bilkent. I
was on the
board of
IEEE, I
worked as
a Python
assistant, I
did an in-
ternship
and a fel-
lowship,
and the
rest is on
my CV.

Actual-
ly I was a
coder be-
fore CS
took over,
starting
with
HTML at
11 and a
lot of
Stack
Overflow.
Now I
vibecode
anyway.
(Okay,
what I re-
ally do is
LLM sys-
tems engi-
neering,
which
means I
connect
systems

and engi-
neer
them.)

ir-globe
draws
what 198
countries
do to each
other and
it has
been going
for months
without
me touch-
ing it, on
in-
frastruc-
ture that
costs 0
dollars a
month.
stitchu
takes a
photo-
graph of a
garment
and gives
back a
sewing
pattern,
and it has
a gate
that de-
cides
whether
that pat-
tern can
actually
be sewn,
which I
wrote af-
ter print-
ing pat-
terns that
passed
every test

I had and
could not
be sewn at
all. lu-
lumelon
measures
what lan-
guage
models say
about a
brand and
prints a
range
rather
than a sin-
gle num-
ber, be-
cause on
its first
live run
the honest
interval
ran from 0
to 33,3.
rabadon is
a gate sys-
tem for
agents,
written in
C++ with
no depen-
dencies,
and it ex-
ists be-
cause an
agent once
told me it
had fin-
ished a job
it had not
finished.

Each of
those
started as
something
I got

wrong,
then
found,
then fixed,
and the fix
is what
this book
hands
over.

How
should
you read
this
book?

The order
is seman-
tic, and I
spent a
full week
reading
and think-
ing before
I settled
it. I start-
ed from
localhost,
because
that is the
joke I kept
seeing.

Writing it
made me
realise the
link only
fails if you
believe a
page is a
place, so
that sub-
ject had to
come first.
Writing

that one
made me
realise I
was talk-
ing about
files with-
out ever
saying
what a
folder is,
so the ter-
minal
came next.

Localho
st then re-
quired
npm run
dev, be-
cause once
you know
whose ma-
chine an-
swers you
want to
know what
is running
on it.
That one
required
npm run
build, be-
cause the
thing you
would put
on another
machine is
not the
thing run-
ning on
yours.
After that
came the
backend,
because a
program
on another

machine is
what the
whole
book has
been walk-
ing to-
wards.
The back-
end re-
quired
data, be-
cause a
backend
keeps
something.
Data re-
quired
.env files
and API
keys, be-
cause the
moment
something
is kept,
whatever
reaches it
matters.

Those
chapters
are
grouped
into five
parts. 101
is what
you are
actually
holding.
201 gets
your
project off
your own
computer.
301 is
where real
users and
real at-

tackers arrive, 401 is the part people touch, and 501 is money. Afterward s I will write about LLM system design, orchestration, loops, workflows and gates.

Every chapter opens with a real incident and closes with the thing you can do tomorrow, on your own project. I don't want to stretch the schedule because 2 months in tech = 10 years of development in any topic.

By the way, new tabs keep opening in

my head,
and a
chapter
about one
thing
wakes up
an idea
about
something
else entirely.
Those
ideas do
not belong
inside the
chapter, so
they go in
the appendix,
and terms
and small
technical
things
that do
not fit
there go in
the foot-
notes.
Both sit at
the bottom
of the page,
so anything
marked
with a star
has a note
waiting for
it.

The appendix
is also where
the writing
about product
management
and the start-

up world
will go.
Companies
still run
events to
hand out
the kind of
knowledge
you can
only get
through
experi-
ence, and
then they
turn cre-
ative peo-
ple away
with rea-
sons on
the level
of you are
a Gemini,
that is
why you
were elimi-
nated. To
tell you
the truth,
I do not
think any-
body is as
well inten-
tioned as I
am, and
people
hold a
strange
principle
about not
sharing
good
things.
You can
steal my
ideas.
There is

sand in
the sea
and there
are ideas
in me, so
passing on
what I
know is
nothing
but a hap-
piness for
me.

Thanks

Years ago
the yazbel
documen-
tation
made me
interested
in comput-
er science,
and I
changed
my de-
partment.
Even
though we
have not
been writ-
ing code
by hand
for the
last year,
it gave me
a love of
CS that
no under-
graduate
degree
could have
given me.
We used
to shorten

loops with
algorithm
analysis,
and now
we do the
same thing
inside
workflows.
The lan-
guage
changed
and the
tool
changed,
while the
purpose
and the
logic did
not, so I
still think
this mo-
tion in CS
is a matter
of
perspec-
tive.

So
when I am
no longer
a sweet
student
but a busi-
ness-
woman, I
hope this
book will
have en-
lightened
somebody,
enter-
tained
somebody,
or made
somebody
think.

Two
heads are
better
than one,
so whatever
work
you are in,
try your
own idea
too. My
endless re-
spect to
everyone
who is
dreaming
and work-
ing for
something.

* Is CS
dead?
The
claim
is
usual-
ly
made
about
a lan-
guage
, not
about
the
field.
COB
OL
was
going
to be
re-

place
d by
C++,
C++
by
Java,
Java
by
Pytho
n,
and
every
one of
them
is still
run-
ning
some-
where
right
now.
What
chang
ed
each
time
was
the
height
of the
lan-
guage
, not
wheth
er
some-
body
had
to de-

cide
what
to
build.

what
is
the
thin
g on
your
scree
n?

A page
is not a
place.
It is a
delivery
that
happens
again
every
time
somebody
looks.

The first
thing you
do with a
coding
agent is
ask it for a
website,
and a
minute
later there
is one.
The termi-
nal prints
an ad-
dress. You
open it

and the
thing you
described
is sitting
in the
browser
looking
finished.

Ask
where that
thing lives
and the
answer
comes
back as in
the brows-
er or on
my com-
puter or
on the in-
ternet. All
three feel
fine right
up until
something
breaks,
and then
none of
them tell
you where
to look.¹

1
**A book
note.** A
model takes
the easiest
road. It
gave you lo-
calhost
with no
backend,
because
that is the
shortest

way to
something
that runs.

What ar-
rives
when a
page
opens?

Make a
folder any-
where you
like and
put a sin-
gle file in
it called
index.html
with one
line inside.

```
<h1>hello
```

Open a
terminal
in that
folder and
run
python3 -
m
http.server
8000. Go
to local-
host:8000
in your
browser.

The word
hello is
there in
large let-
ters, arriv-
ing from a
server
small

enough
that nothing
can
hide inside
it.

Now
right click
on the
page and
choose
view
source.

What
comes up
is the line
you typed,
character
for
character.

Change
the h1 to
a p and
refresh.

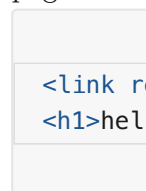
The same
word
comes
back
small.

Nobody
sent you a
smaller
word. You
changed
one instruction
and the
browser
drew the
letters differently,
because
what travelled to it
was never
a page. It
was a set

of instructions for a program that carries them out.

How does the browser learn about the second file?

One file is not how anything real arrives. Next to index.html make a file called style.css containing `h1 { color: crimson; }` and mention it from the page.



Refresh and the word is red.

Your browser had no

way of
knowing
that
style.css
existed.
The only
file it had
been given
was in-
dex.html.
It read
that file
and came
to a line
mention-
ing a
stylesheet.
Only then
did it go
back to
the server
for a sec-
ond file.
The page
tells the
browser
what else
to fetch,
one dis-
covery at
a time.

Open
the devel-
oper tools
with Cmd
and
Option
and I on a
Mac or
F12 on
Windows.
Click the
Network
tab and
refresh
with the

panel
open. Two
rows come
up, in-
dex.html
first and
style.css
second,
and they
can never
come in
the other
order.

A slow
page is
rarely one
slow file.
The docu-
ment ar-
rives and
mentions
something.
That
something
arrives
and men-
tions two
more.
Nothing
further
down the
chain
starts un-
til the
thing be-
fore it has
been read.

Where
does the
page go
when the

server
stops?

'The page
is on my
computer,
so it will
still be
there,' you
may be
thinking.
In the
Network
panel
there is a
checkbox
marked
Disable
cache.
Tick it so
the brows-
er stops
keeping
copies of
its own.
Now go to
the termi-
nal and
stop the
server
with Ctrl
and C,
then re-
fresh the
page.

The
page is
gone and
there is no
error
about
your
HTML,
because
your
HTML

never arrived.
Nothing
had been
sitting in
the browser
waiting
for you.

Start
the server
again and
refresh.
Hello
comes
back in
red exactly
as before,
drawn
from nothing.
A
page is a
delivery
that happens
again every
time somebody
looks.

What
does this
look like
in a real
project?

Open the
project
you have
been
working
on and re-
fresh it
with the
panel

open. The same list appears, only far longer. The document sits at the top. Everything the document mentioned sits under it, and under those sits everything they mentioned in turn.

At the bottom of the panel there is a total size. Every visitor downloads that much before they see anything at all, out of their connection and their phone plan.

That leaves one question for the rest of the book. Some program on some ma-

chine sent
those files.
On the
hello page
you know
which one,
because
you start-
ed it your-
self and it
is still sit-
ting in
your ter-
minal. On
your real
project
you have
probably
never
thought
about it.

CHE
CK

Op
en
the
Ne
tw
ork
pa
nel
on
the
pro
jec
t
yo
u
are
pro

ud-
est
of
an
d
re-
fre
sh
it.

Ho
w
ma
ny
file
s
ar-
riv
ed?

Ho
w
ma
ny
kil
ob
yte
s
did
the
y
co
me
to?

W
hic
h
file
in
tha

t
list
can
yo
u
not
ac-
cou
nt
for
?
Th
ere
is
nea
rly
al-
wa
ys
one
.
Us
ual
ly
a
fon
t
yo
u
sto
pp
ed
us-
ing
mo
nth
s
ago
or
a

li-
bra-
ry
tha
t
ar-
riv
ed
at-
tac
he
d
to
so
me
thi
ng
els
e.
No
thi
ng
an-
no
un
ced
it,
be-
cau
se
it
wa
s
me
nti
one
d
by
a
file

that
it
was
self-
mentio-
ned
by
a
file
.

No
thing
needs
re-
moving
today.
It
has
been
going
out
to
every
visitor
since

the
da
y
it
ap-
pe
are
d.

☐ my
project
passes
this
check

You just
started a
server in-
side a
folder
without
my having
said what
a folder is,
and that
gap has to
close be-
fore we
can ask
whose ma-
chine your
files come
from. So
the termi-
nal and
the folder
come next.

why can your de- ploy not find your files?

The same
command
run one
level up
serves
nothing.
Most de-
ploy
failures
start
here.

You ran a
command
inside a
folder a
moment
ago and a
server ap-
peared,
and I nev-
er said
which
folder it
was read-
ing or why
that mat-

tered. The
same com-
mand run
from one
step high-
er up
would
have
served
nothing.

A
whole
family of
failures
comes out
of that.
The build
that works
on your
machine
and re-
turns file
not found
on the de-
ploy is
one, the
image that
loads lo-
cally and
404s in
production
is another.
In every
one of
them a
program is
standing
somewhere
you did
not ex-
pect, read-
ing a path
that
means
something

different
from
there.

Where
are you
standing?

Open a
terminal.
On a Mac
it is called
Terminal.
On
Windows,
use the
one your
develop-
ment tools
installed
rather
than the
old black
box.

A com-
mand nev-
er runs in
general. It
runs while
you are
standing
somewhere
and where
you are
standing
changes
what it
does.

```
pwd
```

Print
working
directory.

It answers with the absolute path of where you are standing right now.

```
ls -F
```

Plain `ls` lists files and folders together with no way to tell them apart, and `-F` puts a slash on the end of every folder name. Do not try to work it out from the name. `README` has no dot and is a file, while a folder called `backup.old` has one and is still a folder.

```
cd Desktop
```

Change directory, into the Desktop

that sits
inside
wherever
you al-
ready are.
Run `pwd`
again and
the answer
has grown
by one
step. Run
`ls -F` and
the con-
tents have
changed,
because
the same
command
means
something
different
here.

A screenshot of a terminal window with a light gray background. A single line of text, `cd ..`, is displayed in a light blue font and is enclosed within a light blue rectangular box. The box has a thin border and is centered horizontally within the terminal's input area.

Two dots
is a real
entry that
exists in-
side every
folder on
the disk
and it
points at
that fold-
er's
parent.

That is
the whole
of naviga-
tion.
Three
commands
and one
conven-
tion, and

they work
the same
way on
every Mac
and every
Linux ma-
chine and
inside the
terminal
your
Windows
tools
installed.

Is the ter-
minal a
different
place?

If the ter-
minal still
feels like a
second
machine
hiding un-
derneath
the one
you use,
open
Finder or
Explorer
and go to
your
project
folder. Put
that win-
dow on
one side of
your
screen and
the termi-
nal on the
other.
Now `cd`

your way
to the
same folder
and run
ls -F.

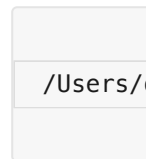
They
are the
same
thing. The
icons on
the left
and the
names on
the right
are one
folder de-
scribed in
two ways,
and the
three com-
mands you
just
learned
will reach
any folder
on the
disk.

What is a
path?

A folder
holds files
and other
folders
and those
hold more.
All of it
hangs
down from
one start-
ing point,
so your

whole disk
is a single
tree.

A path
is direc-
tions
through
that tree
and pwd
printed
one of
them.



It means
start at
the top of
the disk
and go
into Users,
then into
damla and
Desktop
and
project.

Each slash
is a door
you walk
through.

A postal
address is
written
the same
way and a
path is
that writ-
ten on one
line.

There
are two
kinds.



```
/Users/c  
style.cs
```

The absolute one begins with a slash and is measured from the top of the disk, so it means the same file wherever you use it. The relative one is measured from where you are standing, so those same characters point at different files depending on where they are read.

Why is this so hard to learn?

Go back to the folder with `index.html`

and
style.css in
it and
start the
server
again. The
page loads
and the
word is
red. Now
stop the
server, run
cd .. to
step up
one level,
and start
it again
from
there.

```
cd ..  
python3
```

Open lo-
calhost:80
00. There
is no page,
only a list
of folder
names.

Nothing
was moved
and noth-
ing was
deleted.

The href
inside your
HTML
still says
style.css
and it still
means the
style.css
next to
me, and
the server

is now
standing
one level
up where
no such
file sits.
Your
whole
project is
intact and
unreach-
able.

That is
the entire
failure,
and here is
why it
stays in-
visible un-
til it costs
you a day.
A few
years ago
lecturers
in physics
and engi-
neering
started
running
into some-
thing they
could not
explain.

They
would ask
a class to
open a file
from a
particular
directory.
We as stu-
dents usu-
ally strug-
gle to do
exactly
that, my-

self included,
ed, even
though we
use a com-
puter all
day and
use it well.

A file
lives in the
app or on
the com-
puter, and
the way
you reach
it is to
search for
it.

That
model
works for
daily life.
Put every-
thing in
one pile
and search
when you
need
something.
You will
find your
holiday
pho-
tographs
every
time. It
stops
working at
the deploy,
because
the ma-
chine you
deploy to
has no
search
box. It is
handed a

path and
goes ex-
actly
there. If
nothing is
there it
stops.

Directo-
ry struc-
ture is so
ordinary
to the
people
who know
it that
words for
it are hard
to find, so
nobody
explains it
and no-
body asks.

Why does
this de-
cide
whether
things
work?

Every
build step,
every de-
ploy tool
and every
error mes-
sage you
will read
for the
rest of
your life
assumes
you know

where you
are
standing.

File
does not
exist al-
most al-
ways
means a
relative
path was
read from
a folder
you were
not ex-
pecting.

The file
was there
all along,
and the
program
was stand-
ing some-
where else
when it
went look-
ing for it.
You saw
exactly
that a
minute
ago with a
server one
level too
high.

Someth-
ing works
on your
machine
and dies
on the de-
ploy for
the same
reason.
You typed
the com-

mand
while
standing
inside the
project
folder.

The other
machine
runs it
while
standing
wherever
it happens
to start, so
the rela-
tive path
points at a
place that
does not
exist.

Chapter
16 is
where this
bites
hardest.

MIT
teaches a
free course
for exactly
this gap
called The
Missing
Semester
of Your
CS
Education.
Julia
Evans
draws
comics
about the
shell that
are the
friendliest

thing on
the
subject.

CHE
CK

Op
en
a
ter
mi-
nal
an
d
lea
ve
yo
ur
ed-
ito
r
clo
sed
.

Ge
t
to
yo
ur
pro
jec
t
fol
der
us-
ing
onl
y

pw
d,
ls -
F
an
d
cd.
Re
ad
wh
at
co
me
s
bac
k
af-
ter
eac
h
ste
p
in-
ste
ad
of
typ
ing
the
wh
ole
pat
h
fro
m
me
mo
ry.

When you arrive, which folder in that list can you not describe the contents of? There will be one, usually only one

—
mo
dul
es
or
dis
t
or
.ne
xt.

Go
int
o it
an
d
loo
k,
cha
ng-
ing
not
hin
g.
Th
e
ne
xt
tim
e
so
me
thi
ng
bre
aks
tha
t
fol
der
is

going to appear in the error message (which should not be the first time you see the name).

☐ my project passes this check

Next
comes localhost,
and the
reason a
link that
opens in-
stantly for
you does
not open
for your
friend.

why
does
your
link
only
open
for
you?

Your ma-
chine
answers
its own
name.

Your
friend's
machine
answers
its own,
and
finds
nothing.

This chap-
ter is
about fa-
mous lo-
calhost:30
00. Jokes
about it
have been
going
around the
internet
for a long
time.
Many peo-
ple believe

CS is dead
because
they built
a website
hosting on
local. You
click your
own link
and it
opens.
Your
friend
clicks it
and it
does not.
Why does
it happen,
what is lo-
calhost:30
00, and
how to fix
this issue?

What
does lo-
calhost
mean?

It begins
with local-
host:3000
or
127.0.0.1:3
000, which
are the
same ad-
dress writ-
ten two
ways. lo-
calhost is
a name
and
127.0.0.1
is the

number
the name
stands for.
RFC 6761
reserves
both for
one purpose,
and that purpose
is to mean the
computer that is
asking.
When you
type it,
your machine
sends the
request to
a program
on itself.
The request
never leaves
the machine,
which is
why this is
called the
loopback
address.
When
your
friend
types it,
their machine
sends the
request to
a program
on itself
and finds
none.
Each computer's lo-

calhost is
a different
server.

The
number af-
ter the
colon is
the port.
If the ad-
dress is
the build-
ing, the
port is the
apartment
inside it.

One ma-
chine runs
many pro-
grams and
each waits
at its own
number.

3000 is a
convention
develop-
ment tools
picked,
nothing

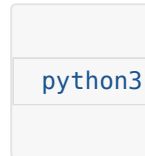
more. The
port is
fine. The
building is
the prob-
lem, be-
cause you
gave your
friend the
name

every
building
uses for
itself.

Can your
own

phone
open it?

Open the
terminal
and cd
into any
folder that
has a file
or two in
it. Run
this.

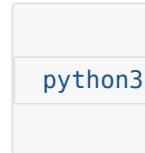


This starts
a small
web
server.

The flag
tells it to
answer
only the
machine it
is running
on. Open
localhost:8
000 in the
browser
and you
see the
folder's
files listed.
Now type
the same
address
into your
phone's
browser.
Nothing
comes
back, even
though the
phone is
on the

same wifi.
The phone
went look-
ing for a
program
at port
8000 on
itself.

Now
stop the
server
with
Ctrl+C
and start
it again
without
the flag.



The flag
was doing
all the
work.

127.0.0.1
told the
server to
listen on
the loop-
back ad-
dress
alone, so
requests
arriving
over wifi
were never
answered.

Without
the flag
this tool
listens on
0.0.0.0,
which is
not an ad-
dress you
connect

to. It means every network interface the machine has, including the wifi card. A request from the phone is now accepted the same as one from the machine itself. You will see 0.0.0.0 again in Docker logs and in cloud deploy output and it always means the same thing.

Now find the laptop's address on the local network. On a Mac that is `ipconfig getifaddr en0` and on Linux `hostname -I`. It looks like

192.168.1.
24. If the
Mac com-
mand
comes
back emp-
ty your
wifi is on
another
interface,
so try en1.
Type
`http://192
.168.1.24:8
000` into
the phone
and the
file list ap-
pears.
Nothing in
the files
changed,
only which
doors the
program
answers
and which
machine
the phone
was told
to look at.

How far
does the
local net-
work
reach?

'So I drop
the flag
and I am
live,' you
may be
thinking.

What you have is a server reachable from the same wifi. 192.168.1.24 is an address inside your home network and means nothing outside it, so a friend in a cafe or a customer on mobile data cannot reach it. And when the laptop lid closes, nothing is running anywhere.

A product has to answer while you are asleep, so the program has to run on a machine that stays on and has an address the internet can route to. Putting it there is

called de-
ploying,
and that is
chapter
16. ir-
globe an-
swers at
three in
the morn-
ing be-
cause it is
not on my
laptop.

CHE
CK

Lo
ok
at
the
ad-
dre
ss
bar
of
the
pro
jec
t
yo
u
are
pro
ud-
est
of.
If
it
be-
gin

s
wit
h
lo-
cal
hos
t
or
127
.0.
0.1
,
the
nu
mb
er
of
pe
opl
e
wh
o
can
op
en
it
is
1.
Th
en
clo
se
the
lap
top
.
Tu
rn
off
wif

i
on
yo
ur
ph
one
so
yo
u
are
on
mo
bil
e
dat
a
an
d
try
to
op
en
the
pro
jec
t.
W
hat
eve
r
co
me
s
bac
k
is
the
an-
sw
er

to
wh
eth
er
it
ex-
ists
yet
.

☐ my
project
passes
this
check

After that
come the
two com-
mands you
run with-
out read-
ing them.
Only one
of npm
run dev
and npm
run build
makes a
thing you
can
deploy.

what
is
the
dif-
fer-
ence
be-
twee
n
npm
run
dev
and
npm
run
build
?

One
builds
pages in
memory
for you.
The oth-
er
writes
the
folder
your
users
get.

Localhost
turned out
to be a
name your
machine
uses for it-
self. The
other half
of the ad-
dress is
still unex-
plained,
because
something
is answer-
ing at that
door, and
in a
framework
project
that some-
thing is
npm run
dev.

You
have run
that com-
mand a
thousand
times
without
wondering
what it is.
Sitting
next to it
in the
same file
there is
npm run
build. It
takes
much
longer and
produces a
folder you
have prob-

ably never
opened.
That fold-
er is the
part your
users get.

What is
running
when you
run npm
run dev?

What runs
is a pro-
gram sit-
ting in
your ter-
minal and
waiting to
be asked
for things,
the same
way
python3 -
m
http.server
did in
chapter 1.
This one is
far clever-
er about
what it
hands
over.

Go to
the termi-
nal where
it says
ready and
press Ctrl
and C.
Refresh
the brows-

er. The
site is
gone and
the error
is a re-
fused con-
nection,
which
means
nothing is
listening
at that
door any
more.

Now
start it
again and
open the
project
folder,
looking for
the HTML
that ar-
rived in
the brows-
er. There
is no such
file. Your
project
has a src
folder full
of compo-
nents and
not one of
them is a
page.
Nothing
on the
disk
matches
what you
saw in
view
source.

The
dev server
built that
page in
memory
the mo-
ment your
browser
asked for
it and
then let it
go. Its
whole job
is to read
the source,
work out
what the
browser
needs
right now
and hand
it over.
Then it
watches
the files so
it can do
the same
thing the
instant
you save.
Nothing
was writ-
ten down,
so a save
appears in
the brows-
er in un-
der a
second.

What
does the

build do?

```
npm run
```

It takes a while, where the dev server started instantly. When it finishes there is a new folder called `dist` on Vite and `.next` on Next.js. Go in there with the commands from chapter 2.

```
cd dist  
ls -F
```

The HTML that did not exist a minute ago is in there. So are the scripts, with names like `index-4f3a9c.js`. Those characters in the

middle are
there so a
browser
can safely
keep an
old copy
without
mistaking
it for the
new one
after you
deploy.
The CSS
from every
component
in the
project
has been
gathered
into one
file.

Your
source is
written in
whatever
shape lets
you work.
This folder
is written
for
browsers,
in one
shape and
once. The
dev server
had been
standing
in for it on
demand
without
ever writ-
ing a word
of it to
disk.

Which
one gets
deployed?

The folder
gets de-
ployed,
never your
source and
never the
dev server.
Deploying
means
building
that folder
and
putting its
contents
somewhere
that an-
swers re-
quests all
day, so a
visitor is
handed a
finished
file instead
of waiting
for a pro-
gram to
work one
out.

You
can serve
that folder
yourself
right now.

```
cd dist  
python3
```

Open lo-
calhost:80
00. That is
your site,

coming
out of a
plain file
server that
has never
heard of
your
frame-
work. It is
the same
server that
handed
you one
line of
HTML in
chapter 1.

Why do
the two
behave
different-
ly?

'Same
code, same
project, so
it will be-
have the
same way,'
you may
be think-
ing. Open
one of
those
scripts in
dist and
read it.
Your func-
tion names
are gone,
replaced
by single
letters.
The spaces

are gone.
The whole
file is one
line.

The
dev server
is opti-
mised for
you. It
starts in-
stantly
and re-
freshes in-
stantly,
keeps your
variable
names so
errors stay
readable,
and
squashes
nothing.

The build
is opti-
mised for
the visitor,
so it
makes the
smallest
files it can,
squashes
everything
together,
drops code
that noth-
ing uses
and short-
ens every
name it
can reach.

Squashi
ng changes
behaviour.
Code that
quietly re-
lied on a

function
still hav-
ing its
own name
breaks, be-
cause the
name is
gone and
you just
read the
file where
it went
missing. A
value read
while the
build is
running
gets frozen
into the
file, so
changing
it on the
server af-
terwards
changes
nothing
until you
build
again, and
chapter 13
is about
the trou-
ble that
causes. A
file the
dev server
found by
walking
your folder
is missing
from the
output en-
tirely if
nothing

imported
it
properly.

So a
project
can run
beautifully
for weeks
and fall
over on
the first
deploy
without a
line of
source
having
changed.

When
somebody
says it
works on
my ma-
chine they
are usually
telling the
truth, be-
cause their
machine
was run-
ning the
friendly
version.

Before
you de-
ploy, run
the build
and serve
the output
locally the
way you
just did,
then click
around
your own
site.

Anything

broken in
that folder
is broken
in produc-
tion, and
you get to
find it
while you
are sitting
in front of
it.

CHE
CK

Ru
n
np
m
run
bui
ld
on
yo
ur
ow
n
pro
jec
t
an
d
let
it
fin-
ish.

Ho
w
lon
g

did
it
tak
e?
W
hat
eve
r
tha
t
nu
mb
er
is,
it
ha
pp
ens
aga
in
on
eve
ry
de-
plo
y.

W
hat
is
in-
sid
e
the
fol
der
?
Go
in
wit
h

cd
dis
t
or
cd
.ne
xt
an
d
run
ls -
F,
the
n
fin
d
the
HT
M
L
yo
ur
sou
rce
ne
ver
con
tai
ne
d.

Th
en
ser
ve
it
wit
h
pyt
ho
n3

-m
htt
p.s
erv
er
800
0
an
d
use
yo
ur
ow
n
site
co
mi
ng
out
of
a
file
ser
ver
tha
t
kn
ow
s
not
hin
g
ab
out
yo
ur
fra
me
wo
rk.

If
so
me
thi
ng
wo
rks
un-
der
np
m
run
de
v
an
d
not
in
the
re,
yo
u
ha
ve
fou
nd
a
rea
l
bu
g
mo
nth
s
be-
for
e a
str
an
ger

wo
uld
ha
ve
fou
nd
it
for
yo
u.

☐ my
project
passes
this
check

Chapter 5
goes to
the shop
front and
the till.
What a
backend
is, and
why some
things can
never live
in the files
you just
looked at
however
well you
hide them.

why
does
the
till
stan
d in
a
dif-
fer-
ent
roo
m?

Everythi
ng de-
livered
to the
browser
belongs
to who-
ever is
standing
there.

Everythin
g so far
has hap-
pened in
one room.
Files ar-
rive and
the brows-
er draws
them. You

have now
read your
own
project
the way a
visitor
reads it. A
shop has a
second
room be-
hind the
counter
where the
till and
the keys
are, and
nobody
has to be
told why.
In a
project
both
rooms
look like
folders sit-
ting next
to each
other in
the same
editor, so
the till
ends up
out on the
shop floor.

Which
room is
the
browser?

The
browser is
the front
one and

everything
in it be-
longs to
whoever is
standing
there.

Every
file that
arrived is
listed in
the
Network
panel and
you can
click any
of them
and read
it, includ-
ing the
scripts you
wrote.

That will
not be
patched
one day,
because it
is what
delivery
means.

'My
code is
minified,
so nobody
can read
it,' you
may be
thinking.
Open one
of your
own
scripts in
that panel
and find
the button
marked
with

braces at
the bot-
tom of the
view. The
one line
becomes
readable
code
again.
Squashing
a script
makes the
file smaller
and it
hides
nothing.

What is a
backend?

A backend
is a pro-
gram on
another
machine
that an-
swers re-
quests. It
runs some-
where else
and listens
at a door.
You talk
to it by
sending
exactly
the kind of
request
you
watched
arrive in
chapter 1.

The
customer
orders and
the
kitchen
cooks. The
customer
does not
come into
the
kitchen,
because
the
kitchen is
a different
room. A
script run-
ning in the
browser is
a locked
cupboard
standing
in the din-
ing room,
and the
customer
has all
evening.

What is
an API?

An API is
the menu.
A menu
tells you
what you
may order
and in
what
form. An
API tells
you which
requests

the
kitchen ac-
cepts,
meaning
this ad-
dress with
this
method
and these
fields.
Anything
not on the
menu is
refused.
Writing
one is the
act of de-
ciding
what
strangers
may ask
you for.

The
verbs on
that menu
are the
methods.
GET
fetches
something,
POST
sends
something
new, PUT
replaces,
DELETE
removes.
A GET is
under-
stood
every-
where to
only fetch
and search
crawlers
act on

that un-
derstand-
ing. That
is how a
link which
deletes a
row gets
triggered
by a
crawler
that was
only look-
ing
around.

What do
the num-
bers
mean?

Every an-
swer
comes
back with
a status
number
and MDN
carries the
definitions
if you
want them
properly.

200
means it
worked.
404 means
the
kitchen
under-
stood you
perfectly
and has no
such dish.
500 means

the
kitchen
broke
while
cooking,
so the
fault is on
their side.

The
two that
get con-
fused con-
stantly are
401 and
403. A 401
means the
kitchen
does not
know who
you are,
while a
403 means
it knows
exactly
who you
are and re-
fuses any-
way.

Identity
and per-
mission
are sepa-
rate sys-
tems with
separate
fixes, and
telling
them
apart is
the whole
of chapter
26.

Can you
break

your own
rule?

Find
something
in your
own
project
that the
page refus-
es to let
you do. A
submit
button
that stays
disabled
until the
form is
valid, a
field with
a maxi-
mum
length, a
section
that only
appears
when you
are logged
in.
Anything
the page
decides.

Open
the devel-
oper tools
and go to
the
Console
tab. Then
type one
of these,
matching
whichever
rule you
found.



The button is live.
The field takes as much as you want.
You did not find a hole in your code, you re-typed a line of it, and every visitor has the same console you just used.

That is why a check written into your page is a suggestion.
Your script runs on their machine.
A check that holds has to run in a room they cannot reach, which is the room this chapter is about.

Where
does the
key go?

Into the
kitchen
and never
into a re-
quest the
customer
can read.

stitchu
sends a
photo-
graph to a
vision
model and
that mod-
el charges
per call.
The key
that pays
for it be-
longs in
the till, so
the brows-
er never
holds it.
The pho-
tograph
goes to a
Cloudflare
Worker,
the
Worker
holds the
key and
calls the
model,
and the
answer
comes
back.
Anyone
can open
the

Network
panel on
stitchu
and read
every re-
quest that
leaves
their ma-
chine and
find noth-
ing in
them to
take.

The
wrong ver-
sion is one
line of dif-
ference.

The
browser
calls the
model di-
rectly with
the key at-
tached to
the re-
quest. It
works on
the first
try (which
is the
trouble)
because
from be-
hind the
counter
both ver-
sions look
the same
to you.
Every visi-
tor now
holds a
working
key billed
to your ac-

count, and
chapter 31
is about
the size of
that bill.

CHE
CK

Op
en
yo
ur
ow
n
pro
jec
t
wit
h
the
Ne
tw
ork
pa
nel
op
en
an
d
use
it
un-
til
re-
qu
est
s
sta
rt

ap-
pe
ar-
ing
.

W
her
e is
eac
h
re-
qu
est
go-
ing
?

Re
ad
the
ad-
dre
ss
of
eac
h
one
. If
it
goe
s
str
aig
ht
to
so
me
bo
dy
els
e's

ser
vic
e
the
n
yo
ur
bro
ws
er
is
tal
kin
g
to
tha
t
ser
vic
e
di-
rec
tly
an
d
eve
ry-
thi
ng
in
tha
t
re-
qu
est
is
pu
bli
c.

What
in
your
code
costs
money
or
grants
access?
An
API
key
, a
database
URL,
an
admin
token
. What
here
does

it
live
rig
ht
no
w?

If
it
rea
che
d
the
bro
ws
er
the
n
it
is
pu
bli
c,
an
d
the
ne
xt
tw
o
cha
pte
rs
are
ab
out
mo
vin
g
it.

☐ my
project
passes
this
check

Chapter 6
goes to
where
data lives,
and why a
file works
perfectly
until two
people use
your
project at
the same
time.